



Python程序设计

一、基础知识

学习内容：

1. Python代码编写规范
2. 字符串与更多的输出选项
3. Python编程快速入门

大纲

- Python语言概述和安装
- Python基础知识：基本的输入输出
- Python基础知识：变量和对象
- Python基础知识：数字类型
- Python基础知识：模块和包
- Python代码编写规范
- Python基础知识：字符串与更多的输出选项
- Python编程快速入门

1.5 Python代码编写规范

PEP 8 -- Style Guide for Python Code

<https://www.python.org/dev/peps/pep-0008/>

Google Python Style Guide

<http://google.github.io/styleguide/pyguide.html>

(1) 缩进

- python程序是依靠代码块的缩进来体现代码之间的逻辑关系的
 - 代码块的开始：类定义、函数定义、选择结构、循环结构等，行尾出现: --行尾的冒号表示缩进的开始
 - 缩进结束表示一个代码块的结束了
 - 同一个级别的代码块的缩进量必须相同

```
with open(fn) as fp:
    for line in csv.reader(fp):
        if line:
            print(*line)
```

- 建议以4个空格为基本缩进单位，不建议采用制表符来缩进
- IDLE可以通过下面的方法进行代码块的缩进和反缩进：

Format → Indent Region/Dedent Region

快捷键为 Ctrl + [和 Ctrl +]

Python代码编写规范 (cont.)

(2) 每个**import语句**只导入一个模块，最好按**标准库**、**扩展库**、**自定义库**的顺序依次导入。

```
import csv
import random
import datetime
import pandas as pd
import matplotlib.pyplot as plt
```

Python代码编写规范 (cont.)

(3) 必要的空格与空行:

- 最好在每个函数、类定义、不同功能的代码块之间增加空行，以提高可读性。
- 在运算符两侧各增加一个空格
- 逗号后面增加一个空格。

```
sockDianming.close()
sockDianming = None

# 开始点名
def buttonStartDianmingClick():
    # 如果还没有注册，拒绝运行
    if int_zhuce.get() == 0:
        tkinter.messagebox.showerror('很抱歉', '请联系作者进行软件注册!')
        return
    if int_zuoye.get() == 1:
        tkinter.messagebox.showerror('很抱歉', '现在正在收作业')
        return
    if int_canDianming.get() == 1:
        tkinter.messagebox.showerror('很抱歉', '现在正在点名')
        return
    tkinter.messagebox.showinfo('恭喜', '设置成功，现在开始点名')
    #开始点名
    int_canDianming.set(1)
    global tDianming_id
    t = threading.Thread(target=thread_Dianming)
    t.start()
    tDianming_id = t.ident
    buttonStartDianming = tkinter.Button(root, text='开始点名', command=buttonStartD
    buttonStartDianming.place(x=20, y=60, height=30, width=100)

def buttonStopDianmingClick():
    # 如果还没有注册，拒绝运行
    if int_zhuce.get() == 0:
        tkinter.messagebox.showerror('很抱歉', '请联系作者进行软件注册!')
```

The diagram illustrates Python code formatting rules with arrows pointing to specific code elements:

- 空行** (Empty line): Points to the empty line between `sockDianming = None` and the function definition, and between the function definition and the `buttonStartDianming` button creation.
- 有空格** (With space): Points to the space around the `def` keyword, the space around the `global` keyword, and the space around the `def` keyword in the second function.
- 没有空格** (Without space): Points to the `buttonStartDianming` variable name, which is split across two lines.

Python代码编写规范 (cont.)

(4) 尽量不要写过长的语句。如果语句过长，可以考虑拆分成多个短一些的语句，以保证代码具有较好的**可读性**。如果语句确实太长而超过屏幕宽度，最好使用**续行符**（line continuation character）“\”，或者使用圆括号将多行代码括起来表示是一条语句。

```
x = 1 + 2 + 3\  
    + 4 + 5\  
    + 6  
  
y = (1 + 2 + 3  
     + 4 + 5  
     + 6)
```

Python代码编写规范 (cont.)

(5) 注释:一个好的、可读性强的程序一般包含30%以上的注释。

- 以**符号#**开始，表示本行#之后的内容为注释

在IDLE开发环境中，可以通过下面的操作快速注释/解除注释大段内容

Format→Comment Out Region/Uncomment Region

快捷键： ALT+3 ALT+4

```
# 读取并显示上面代码生成的csv文件内容
with open(fn) as fp:
    for line in csv.reader(fp):
        if line:
            print(*line)
```

- 包含在一对**三引号'''...'''**或**"""..."""**之间且不属于任何语句的内容将被解释器认为是注释，经常在函数体开始处添加长注释（三引号），作为docstring

```
def isclosed(self):
    """True if the connection is closed."""
    return self.fp is None
>>> print(isclosed.__doc__)
True if the connection is closed.
```

Python代码编写规范 (cont.)

(6) 虽然Python运算符有明确的优先级，但对于复杂的表达式建议在适当的位置使用**括号**使得各种运算的隶属关系和顺序更加明确、清晰。

PS. 代码优化建议

- (7) 在开发速度和运行速度之间尽量取得**最佳平衡**。
- ✓ **内置对象运行速度最快**，标准库对象次之，用C或Fortran编写的扩展库速度也比较快，而纯Python的扩展库往往速度慢一些。
- ✓ 在开发项目时，应**优先使用Python内置对象**，其次考虑使用Python标准库提供的对象，最后考虑使用第三方扩展库。

代码优化建议(cont.)

- (8) 根据运算特点选择最合适的数据类型来提高程序的运行效率。
- ✓ 如果定义一些数据只是用来频繁遍历并且关心顺序，最好优先考虑元组。
- ✓ 如果需要频繁地测试一个元素是否存在于一个序列中并且不关心其顺序，尽量采用集合。
- ✓ 列表和元组的in操作的时间复杂度是线性的，而对于集合和字典却是常数级的，与问题规模几乎无关。

代码优化建议(cont.)

(9) 充分利用关系运算符以及逻辑运算符and和or的惰性求值特点，合理组织条件表达式中多个条件的先后顺序，减少不必要的计算。

(10) 充分利用生成器对象或类似迭代对象的惰性计算特点，尽量避免将其转换为列表、元组等类型，这样可以减少对内存的占用，降低空间复杂度。

(11) 减少内循环中的无关计算，尽量往外层提取。

1.3 Python编程规范与代码优化建议

```
>>> import this
```

```
The Zen of Python, by Tim Peters
```

```
Beautiful is better than ugly.
```

```
Explicit is better than implicit.
```

```
Simple is better than complex.
```

```
Complex is better than complicated.
```

```
Flat is better than nested.
```

```
Sparse is better than dense.
```

```
Readability counts.
```

```
Special cases aren't special enough to break the rules.
```

```
Although practicality beats purity.
```

```
Errors should never pass silently.
```

```
Unless explicitly silenced.
```

```
In the face of ambiguity, refuse the temptation to guess.
```

```
There should be one-- and preferably only one --obvious way to do it.
```

```
Although that way may not be obvious at first unless you're Dutch.
```

```
Now is better than never.
```

```
Although never is often better than *right* now.
```

```
If the implementation is hard to explain, it's a bad idea.
```

```
If the implementation is easy to explain, it may be a good idea.
```

```
Namespaces are one honking great idea -- let's do more of those!
```

```
...
```

1.3 Python编程规范与代码优化建议

- 以动手实践为荣，以只看不练为耻。
- 以打印日志为荣，以单步跟踪为耻。
- 以空格分隔为荣，以制表分隔为耻。
- 以单元测试为荣，以手工测试为耻。
- 以代码重用为荣，以复制粘贴为耻。
- 以多态应用为荣，以分支判断为耻。
- 以Pythonic为荣，以冗余拖沓为耻。
- 以总结思考为荣，以不求甚解为耻。

大纲

- Python语言概述和安装
- Python基础知识：基本的输入输出
- Python基础知识：数字类型
- Python基础知识：变量和对象
- Python基础知识：模块和包
- Python代码编写规范
- Python基础知识：字符串与更多的输出选项
- Python编程快速入门

1.4.4 字符串(str)

- 与其他语言一般用双引号来定义字符串字面值不同，Python可以使用**单引号**和**双引号**
 - Python3的字符串默认为Unicode字符串，从而可以包含中文字符串
 - 如果字符串中有双引号，则使用单引号定义，反之亦然

'abc' "Python程序设计基础" "What's your name?" """Thank you!" she said. '

- 单引号**和**双引号**定义的字符串不能跨越多行，字符串中想要有换行需要使用字符转义
- 三引号**'''或'''表示的字符串可以换行，用于长字符串，也用于在程序中表示较长的注释。

```
>>> a = '''first line
line 2
line 3
last line.'''
```

- 字符串属于**不可变序列**
- 空串表示为""或'''

```
>>> a = '''first line
line 2
line 3
last line.'''
>>> a
'first line\nline 2\nline 3\nlast line.'
>>> print(a)
first line
line 2
line 3
last line.
>>>
```

字符串转义符\

- **转义符**：表示需要后面的字符需要特别对待，不是它本来的意思了

特殊字符	含义	特殊字符	含义
\'	一个单引号	\"	一个双引号
\\	一个\	\t	制表符
\n	换行符	\r	回车

- 如一个字符串内容有单双引号，则需要通过在引号前加上转义字符,表示引号为字符串的一部分，而不是字符串的结束

```
>>> s = 'he said:"I\'m superman."'
```

```
>>> s
'he said:"I\'m superman."'
>>> print(s)
he said:"I'm superman."
```

```
>>> s2 = "No.\tname\tage\t\n1\tTom\t21"
```

```
>>> s2
'No.\tname\tage\t\n1\tTom\t21'
>>> print(s2)
No.      name      age
1        Tom       21
```

转义字符	描述
\\(在行尾时)	续行符
\\	反斜杠符号
\'	单引号
\"	双引号
\a	响铃
\b	退格(Backspace)
\e	转义
\000	空
\n	换行
\v	纵向制表符
\t	横向制表符
\r	回车
\f	换页
\yy或\yyy	八进制数yy代表的字符，例如：'\12' 代表换行
\xyy	十六进制数yy代表的字符，例如：'\x0a'代表换行
\other	其它的字符以普通格式输出
\uxxxx	描述码点xxxx对应的字符，如：'\u56fd' 是'中'

```
>>> a = '\t\106\t\x61\t\c'
```

\106 → F

\x61 → a

\c → \c

```
>>> a = '\t\106\t\x61\t\c'
>>> a
"\tF\ta\t\c"
>>> print(a)
'      F      a      \c'
>>> len(a)
8
>>> str1="hello"
>>> len(str1)
5
>>>
```

原始字符串

```
>>> a = "C:\Users\xiaochunxiao\Desktop"  
SyntaxError: (unicode error) 'unicodeescape' codec can't decode  
bytes in position 2-3: truncated \UXXXXXXX escape
```

```
>>> a = 'C:\\Users\\xiaochunxiao\\Desktop'  
>>> |
```

- 字符串界定符前面加字母r或R表示原始字符串，表示字符串里面的\都不起作用了，即其中的特殊字符不进行转义，但字符串的最后一个字符不能是\。

```
>>> a=r"C:\Users\xiaochunxiao\Desktop"  
>>> a  
'C:\\Users\\xiaochunxiao\\Desktop'  
>>> print(a)  
C:\Users\xiaochunxiao\Desktop
```

```
>>> a=r"C:\Users\xiaochunxiao\Desktop\  
SyntaxError: EOL while scanning string literal
```

交互式console在计算表达式时：如果表达式obj的值不为None时调用`print(repr(obj))`。对于字符串而言，输出结果为转义的字符串

`print(obj)`: 用户友好的输出，对于字符串而言，一串字符

- `repr(obj)`: 内部表示字符串

```
a=r"C:\Users\xiaochunxiao\Desktop"  
print(a)  
print(repr(a))
```

```
>>> a=r"C:\Users\xiaochunxiao\Desktop"  
>>> a  
'C:\\Users\\xiaochunxiao\\Desktop'  
>>> print(repr(a))  
'C:\\Users\\xiaochunxiao\\Desktop'
```

英文字母的ASCII码是连续的
包括小写英文字母a..z、大写英文字母，十进制数字

字符串相关内置函数

转换为字符串
转换为数字类型

函数	含义	实例	结果
bin(x)	把数字x转换为二进制字符串	bin(4)	'0b100'
oct(x)	把数字x转换为八进制字符串	oct(25)	'0o31'
hex(x)	把数字x转换为十六进制字符串	hex(75)	'0x4b'
str(obj)	把对象obj转换为字符串	str(16), str(1/2)	('16', '0.5')
int(x[,d])	把数字x截取为整数，或将基数为d的数字型字符串x转换为整数，如果d没有缺省为十进制	int(3.14), int('314') int('ff',16)	(3,314) 255
float(x)	将对象(数字或字符串)x转换为浮点数	float(3),float('3.14') float('12e-2')	(3.0,3.14) 0.12
len(obj)	返回序列类对象obj包含的元素个数	len('Hello World'),len('')	(11, 0)
ord(s)	返回长度为1的字符串s中的字符对应的Unicode码（ASCII码是其子集）	ord('a'),ord('A'),ord('你') ord('0'), ord('\n')	(97, 65, 20320) (48, 10)
chr(x)	返回Unicode编码为x的字符串 (len=1)	chr(65),chr(10),chr(20320) chr(ord('d')-ord('a')+ord('A'))	('A', '\n', '你') 'D'

字符串支持的运算符 + * %

+ 字符串合并成新的字符串

```
>>> a='abc' + '123'      #生成新对象 'abc123'
```

```
>>> b='\141b\x61' + '123'  # 'aba123', ord('\a')=97, 八进制为141,
    十六进制为61
```

* 字符串和整数相乘相当于字符串的内容重复多次, 整数小于等于0时生成空字符串

```
>>> a='*- ' * 30  # '*****-
*****-'
```

```
>>> a= 4*'NE'     # 'NENENENE'
```

- **format_string % obj** 把对象obj按格式要求format_string转换为字符串。

字符串格式化 %运算符

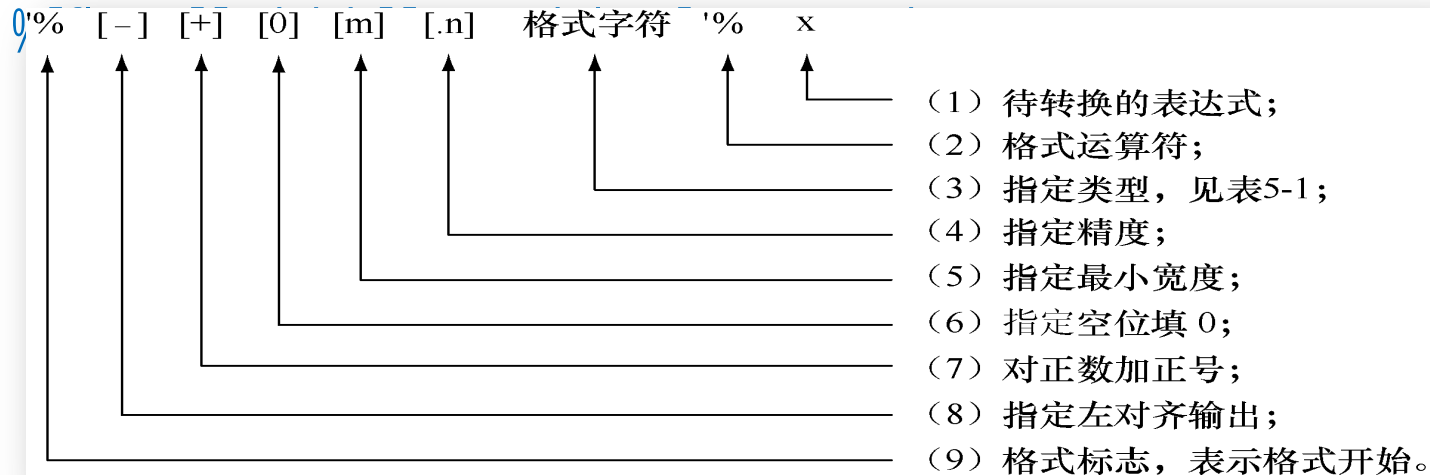
- 类似C语言的printf格式化输出：

format_string % (value1, value2, ...)

把 (value1, value2, ...)中各对象按**格式化字符串format_string**的要求转换为字符串

格式化字符串由固定文本以及格式说明符混合而成

格式说明符：



- flags: 对齐和填充等
 - [align-][sign+][#][0] 次序不规定
 - 默认右对齐。如果为-表示左对齐
 - #标志：type为xXo等时前面加上相应的进制表示前缀，对于浮点类型则要带小数点
- 宽度和精度可选，没有设置时采用系统缺省值

字符串格式化

注意：除了%s对应的值是任意类型外，其他格式字符都要求值为相应的类型

• 常用格式字符——与待格式化的表达式类型匹配

格式字符	说明	格式字符	说明
%s	字符串(采用str()的显示)	%x	十六进制整数
%r	字符串(采用repr()的显示)	%e	指数(基底写为e)
%c	单个字符	%E	指数(基底写为E)
%b	二进制整数	%f、%F	浮点数
%d	十进制整数	%g	指数(e)或浮点数(根据显示长度)
%i	十进制整数	%G	指数(E)或浮点数(根据显示长度)
%o	八进制整数	%%	字符%

字符串格式化例子

```
>>> x = 1235
>>> '%d' % x
'1235'
>>> "%#o" % x
'0o2323'
>>> "%04x" % x
'04d3'
>>> "%e" % x
'1.235000e+03'
>>> '%-6.2f' % 3.1415
'3.14'
```

```
>>> '%-10.2f'%3.4
'3.40'
>>> '%-010.2f'%3.4
'3.40'
>>> '%010.2f'%3.4
'0000003.40'
>>> '%+010.2f'%3.4
'+000003.40'
>>>
```

```
>>> "%s" % 65
'65'
>>> "%10s" % 65
'          65'
>>> "%s" % 65333
'65333'
```

```
>>> "%d" % "555"
Traceback (most recent call last):
  File "<pyshell#19>", line 1, in
<module>
    "%d"% "555"
TypeError: %d format: a number is
required, not str
```

%常用格式举例

```
>>> "%6.2s" % "python lesson"  
,  
py  
>>> "%6.2s" % 65333  
,  
65  
>>> |
```

格式	说明
10.2f	格式化浮点数，总宽度为至少10，四舍五入到小数点后第2位
10.2e	以科学计数法表示浮点数，总宽度至少为10，系数保留到小数点后2位
5d	将整数格式化为总宽度至少为5的十进制形式，不够填充空格
05x	将整数格式化为总宽度至少为5的十六进制形式，不够填充0
05o	将整数格式化为总宽度至少为5的八进制形式，不够填充0
5b	将整数格式化为总宽度为5的二进制形式， 新的format方法支持
50s	将字符串格式化为总宽度至少为50的字符串，不足则补上空格
50.2s	截取字符串的前2位，将其格式化为总宽度为50的字符串，不足补空格
-10.2f	向左对齐格式化对象，默认右对齐
+10.2f	格式化浮点数，如果是正数，则自动加上正号+

```
>>> "%+10.2f" % 3.1415  
,  
+3.14  
>>> "%+010.2f" % 3.1415  
,  
+000003.14  
>>> |
```

练习：字符串格式化例子

```
>>>a = 3.6674
```

```
>>>'%7.3f' % a
```

```
' 3.667'
```

```
>>> "%d:%c"%(65, 65)
```

```
'65:A'
```

```
>>> """My name is %s, and my age is %d"""%( ' Dong Fuguo' , 38)
```

```
'My name is Dong Fuguo, and my age is 38'
```

输出语句print更多选项

```
>>> help(print)
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file: a file-like object (stream); defaults to the current sys.stdout.
    sep: string inserted between values, default a space.
    end: string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.
```

- 内置函数print()用于输出信息到标准控制台或指定文件，语法格式为：
`print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout, flush=False)`
- ✓ sep参数之前为需要输出的内容（可以有多个）；
- ✓ sep参数用于指定数据之间的分隔符，默认为空格；
- ✓ end参数用于指定输出完数据之后再输出什么字符；
- ✓ file参数用于指定输出位置，默认为标准控制台，也可以重定向输出到文件。

```
>>> print(1, 3, 5, 7, sep='\t')           #修改默认分隔符
1      3      5      7
>>> for i in range(10):                   #修改end参数，每个输出之后不换行
    print(i, end=' ')
0 1 2 3 4 5 6 7 8 9
>>> with open('test.txt', 'a+') as fp:
    print('Hello world!', file=fp)        #重定向，将内容输出到文件中
```

大纲

- Python语言概述和安装
- Python基础知识：基本的输入输出
- Python基础知识：变量和对象
- Python基础知识：数字类型
- Python基础知识：模块和包
- Python代码编写规范
- Python基础知识：字符串与更多的输出选项
- Python编程快速入门

1.9 Python编程快速入门

- 问题1：用户输入一个三位自然数，计算并输出其佰位、十位和个位上的数字。

```
x = input('请输入一个三位数：')
x = int(x)
a = x // 100
b = x // 10 % 10
c = x % 10
print(a, b, c)
```

2.5 精彩案例赏析（第二版）

- 还可以这样写

```
x = input('请输入一个三位数：')
x = int(x)
a, b = divmod(x, 100)
b, c = divmod(b, 10)
print(a, b, c)
```

```
>>> divmod(123, 100)
(1, 23)
>>>
```

还可以再简单些吗？

2.5 精彩案例赏析

- 居然可以这样？OMG

```
x = input('请输入一个三位数：')  
a, b, c = map(int, x)  
print(a, b, c)
```

函数式编程



1.9 Python快速入门

- 问题2：已知三角形的两边长及其夹角，求第三边长。

$$c = \sqrt{a^2 + b^2 - 2ab\cos\theta}$$

```
import math
x = input( '输入两边长及夹角(度)，以英文逗号分隔：')
a, b, sita = eval(x)  #序列解包 字符串那章学后就可以理解
c = math.sqrt(a**2 + b**2 - 2*a*b*math.cos(sita*math.pi/180))
print('c=', c)
```

1.9 Python快速入门

- 问题3：任意输入三个英文单词，按字典顺序输出。

```
s = input('x,y,z=')
x, y, z = s.split(',') #字符串那章学后就可以理解
if x > y:
    x, y = y, x
if x > z:
    x, z = z, x
if y > z:
    y, z = z, y
print(x, y, z)
```

```
>>> s = input(' x, y, z=')
x, y, z=boy, like, map
>>> s
'boy, like, map'
>>> x, y, z = s.split(',')
>>> x
'boy'
>>> y
'like'
>>> z
'map'
>>>
```

8.6 使用IDLE调试代码（第一版p202）

- 首先单击菜单 “Debug” ➔ “Debugger” 打开调试器窗口
- 然后打开并运行要调试的程序
- 切换到调试器窗口进行调试

- Go: 继续执行
- Step: 按顺序执行代码，如果为函数，则进入函数体内
- Over: 按顺序执行代码，如果为函数，则不进入函数体内
- Out: 执行代码直到跳出当前函数体
- 在源代码.py文件中鼠标右键弹出上下文菜单后可以设置和清除当前行的断点

IDEL调试器窗口

